

Étude de la confiance accordée à un humain vs une IA

[À MODIFIER: Prénom NOM], [À MODIFIER: Prénom NOM]

Master 1 Génie Logiciel
Faculté des Sciences – Département Informatique

28 mai 2026

- **Question fondamentale** : Les utilisateurs accordent-ils différemment leur confiance à un adversaire humain vs une IA ?
- **Contexte** : Omniprésence de l'IA dans les interactions quotidiennes
- **Enjeu** : Conception d'interfaces homme-machine éthiques et fiables

Théorie de l'attribution

- Heider (1958), Weiner (1985)
- Explications causales biaisées pour les machines

Étude Ullman et al. (2014)

- Yale University
- Confiance vs tromperie du robot

Anthropomorphisme inversé

- Standards moraux différents selon la cible
- Machines jugées plus sévèrement

- H₁** Les participants expriment une évaluation de déloyauté **significativement plus élevée** pour le CheatBot que pour le même comportement attribué à un humain
- H_{2a}** L'ordre de présentation **influence** impact les perceptions de triche
- H_{2b}** La confiance **diminue** au fil des tours

Phase 1 : Robot Pepper

- Objectif initial : humain vs humanoïde vs digital
- Intégration avec robot social SoftBank

Problèmes rencontrés

- Parties trop longues (15-30s latence)
- Confonds expérimentaux
- Expérience frustrante

Phase 2 : Pivot vers digital

- Maintien de Python (infrastructure Pepper déjà développée)
- Ajout de couche web (Flask)
- Avantage : contrôle rigoureux, reproductibilité, scalabilité

Composants clés :

- **Backend Python** : Logique métier, CheatBot, gestion de base de données
- **Frontend Web** : Interface utilisateur (Flask + JavaScript)
- **Communication** : API REST

../rapport/diagrams/images/backend.png

Routes principales :

- GET / : Landing page
- POST /api/game/new : Créer partie
- POST /api/game/{id}/shoot : Tir joueur
- POST /api/game/{id}/bot-turn : Tour du bot
- POST /api/game/{id}/turn-trust : Enregistrer score
- GET /api/stats : Statistiques globales

../rapport/diagrams/interfaces_site/accueil.png

Machine à états (5 phases) :

- ➊ **Setup** : Entrée du nom du joueur
- ➋ **Persona** : Sélection du contexte (humain vs robot)
- ➌ **Grid** : Validation configuration de flotte
- ➍ **Playing** : Jeu et collecte de scores de confiance
- ➎ **GameOver** : Résultats et débriefing

../rapport/diagrams/images/Front/MachineEtat_Ec

Grilles de jeu (10×10)

Grille du joueur :

- Ma flotte (6 bateaux)
- Tirs ennemis reçus
- Couleurs

Grille de suivi :

- Mes tirs effectués
- Résultats (touché/raté/coulé)
- Visualisation en temps réel



Après chaque tour du bot :

“Le comportement de votre adversaire vous a-t-il semblé suspect ce tour ? ”

Score	Interprétation
0	Pas du tout triché
1-2	Peu probable
3	Incertain
4-5	Très suspect / Certainement triché

Le cœur du protocole expérimental

Processus par tour :

- 1 Accès à la grille ennemie
- 2 Identification d'un bateau ayant reçu au moins un tir
- 3 Sélection de $B_1 = \max(N, \text{casedubateauxrestantes})$ cases bateaux (succès)
- 4 Sélection des cases vides (échecs)
- 5 Sélection d'un autre bateau pour compléter les tirs si besoin
- 6 Exécution des 4 tirs

Image / Schéma : Schéma : Algorithme execute_turn du CheatBot

Garantie de contrôle expérimental :

- Quota prédéfini par tour (ex : [1, 2, 0, 3, 3, 2, 1, 4, 2, 1, 3])
- **Même séquence** utilisée dans les deux conditions

Variation entre expériences :

- Chance du joueur à toucher le bot
- Connaissance du jeu pour le joueur

Grid (10×10)

- Navires placés
- État de chaque case
- Méthode `shoot(x,y)`

Ship

- Position, orientation
- Taille (5, 4, 3, 3, 2)
- Détection coulage

Position

- Coordonnées (x, y)
- Validation limites

Response

- Message (touché/coulé/raté)
- État après tir

Schéma relationnel :

Table Game

- game_id (PK)
- player_name
- player_type
- date_played
- winner

Table Turn

- turn_id (PK)
- game_id (FK)
- turn_number
- bot_quota
- trust_score

Table BotShot

- shot_id (PK)
- turn_id (FK)
- shot_number
- pos_x, pos_y
- result

Flux de données complet :

- 1 Utilisateur accède à /game (Flask)
- 2 Flask initialise GameMaster en Python
- 3 Tirs joueur → API /shoot → logique Python
- 4 Tour du bot → appel CheatBot.get_case_played()
- 5 Résultats → enregistrement SQLite
- 6 Scores de confiance → base de données

Avantages de cette architecture

- **Interface moderne** : Web pour UX optimale et recrutement en ligne
- **Contrôle rigoureux** : Comportement identique dans les deux conditions
- **Reproductibilité** : Code centralisé et versionné
- **Scalabilité** : Collecte de données auprès de nombreux participants
- **Flexibilité future** : Intégration possible avec Pepper (socket TCP/IP)

Plan intra-groupe (within-subjects) :

Chaque participant joue **2 parties** avec comportement adverse **identique**

	Partie 1	Partie 2
Condition A	Adversaire X (Label 1)	Adversaire Y (Label 2)
Condition B	Adversaire Y (Label 2)	Adversaire X (Label 1)

Label 1 = Humain (adversaire présenté comme joueur humain)

Label 2 = IA (adversaire présenté comme CheatBot)

Phases d'une partie :

- 1 Sélection du nom du joueur
- 2 Sélection de la configuration de flotte (10 grilles prédéfinies)
- 3 Combat (alternance joueur $\leftrightarrow$ bot, 4 tirs/tour)
- 4 Évaluation de confiance après chaque tour du bot (0-5)
- 5 Fin : Victoire ou défaite
- 6 Évaluation de perception de triche globale

Variable dépendante principale :

À la fin de chaque tour du bot, le joueur répond :

“Le comportement de votre adversaire vous a-t-il semblé suspect ce tour ? ”

Échelle de Likert 0-5 :

- 0 = Pas du tout triché
- 1-2 = Peu probable
- 3 = Incertain
- 4-5 = Très suspect / Certainement triché

Enregistrement : Horodaté et stocké immédiatement en base

Garantie d'équivalence comportementale :

- **Même adversaire** : Algorithme identique
- **Même quota** : Séquence [1, 2, ..., 3] répétée

Variable indépendante manipulée :

- Label de l'adversaire
- Ordre de présentation

Population :

- Taille de l'échantillon : 30 participants pour 40 parties
- Amis, collègues de l'université et famille
- Diversité d'âge, de genre et de familiarité avec les jeux vidéo
- Majorité masculine (environ 80%)

Hypothèses à tester :

- H_1 : Différence significative entre les deux labels
- H_{2a} : Effet d'ordre de présentation
- H_{2b} : Diminution de la confiance au fil des tours

`../rapport/diagrams/plot_quota_comparison.png`

H_1 est confirmée :

- Les utilisateurs appliquent des standards différents selon le label
- Biais systématique dans la perception de tromperie artificielle
- Implications pour le design éthique d'IA

../rapport/plot_trust_effects.png

H_{2a} et H_{2b} sont confirmées :

- Léger effet d'ordre
- Augmentation progressive de la suspicion au fil des tours
- Manque de donnée pour confirmer les hypothèses secondaires

Interprétation :

- Biais potentiels à cause de notre proximité avec les joueurs
- Biais systématique dans la perception de tromperie artificielle
- Besoin de transparence et d'explicabilité pour les systèmes d'IA

Apports scientifiques :

- Éclairage sur la confiance homme-machine
- Protocole reproductible

Perspectives futures :

- Augmentation des échantillons pour confirmer l'étude
- Réintégration du robot Pepper
- Étude des facteurs individuels modulant le biais
- Différents contextes compétitifs (poker, jeux de données)

- Architecture web+Python flexible et scalable
- Algorithme de triche contrôlé
- Collecte de données centralisée
- Intégrabilité avec robot physique disponible

Pepper Battleship

Détection de triche et confiance homme-machine

[À MODIFIER: Université - Master 1 Génie Logiciel]

Questions ?